



Digital Nurture 5.0

Deep Skilling Handbook - Python FSE

Program Highlights

- The Deep Skilling learning program runs for a period of 7 weeks. Go through the recommended self- learning resources and practice the exercises to excel in the recommended Python FSE modules.

Deep Skilling covers the below mentioned skills.

- Design patterns, Data structures and Algorithms | Python Backend Frameworks | Frontend Development Basics | Database Integration | QA Concepts & Test Automation | Unit Testing | Cloud Deployment | Agile, Code Review & Cross-Functional Collaboration | Cloud basics | GenAI fundamentals

Recommended Program Sequence

The learning journey contains the modules mentioned above, followed by a Deep Skilling Final Assessment. It is recommended to follow a sequence of skills learning, the details of which are given below.

Reason for the sequence recommendation:

Any engineering competency landscape can be classified into four major constructs

1. Engineering Concepts

The core concepts for any engineering landscape. This is the foundation on which all programming languages and frameworks are built.

Relevant skills:

- Design Patterns and Principles
- Data Structures and Algorithms

2. Programming Languages

The languages are used to implement the necessary logic for the requirements.

Relevant skills:

- Python programming - Covered in upskilling
- SQL - Basic (covered in upskilling) and Advanced (covered in deep skilling)

3. Products and Frameworks

There are numerous frameworks in the programming family to build applications. These enable the programmers to develop different types of applications by providing a gamut of features provided over a single integrated environment.

Relevant skills:

- Unit testing - PyTest, unittest, Jest & Mocha
- Python Backend Frameworks - Django, Flask & FastAPI
- Angular, React and Vue.js overview
- **Track core skill** - QA Concepts & Test Automation (Selenium Basics)

4. Platforms

These are the enablers as a base to host the application developed using the Engineering concepts, programming languages, product and frameworks to make it accessible by all users, reliable and scalable.

Relevant skills:

- Agile
- Code Review & Cross-Functional Collaboration
- Cloud fundamentals

Apart from all the mentioned skills, the very widely emerging concentration that any industry is looking into is the 'Generative Artificial Intelligence' or the 'GenAI'.

Let us get an introduction to it. You can go through it in the module 'Gen AI Fundamentals'.

Program Completion Criteria

To successfully complete this learning program, candidates must meet the following criteria:

Hands-on Exercises:

- Candidates must complete the hands-on exercises mapped against every skill. These exercises are designed to reinforce learning objectives and ensure a practical understanding of the material.

Final KBA Assessment:

- Candidates must pass the final **Knowledge-Based Assessment (KBA)**. This assessment will evaluate their comprehensive understanding and application of the concepts covered throughout the program.

Learning Approach

DN 5.0 Deep Skilling Program adopts a comprehensive and blended learning approach to ensure an engaging and effective educational experience.

The program comprises two essential learning components:

- **Self-paced learning** through open-source learning reference links.
- **Bi-weekly SME connect** sessions conducted by experts.

Self-Paced Learning using Open-source Reference Links

- Please refer to the learning reference links provided to learn and understand the recommended concepts.

We expect you to dedicate 10-12 hours of focused attention weekly to your learning modules.

Introduction to Cognizant SkillSpring

Cognizant SkillSpring is a multimodal, AI-native, conversational learning platform designed to redefine learning in the AI era and help businesses cultivate AI-ready talent at scale. Moving beyond static courses, SkillSpring uses AI-powered agents to personalize learning, accelerate skill mastery, and embed learning directly into the flow of work.

Candidates are highly encouraged to leverage the SkillSpring platform and the specific courses highlighted in the modules below to enhance your deep skilling journey and become future-ready.

You’d be enrolled to the relevant courses for which you’d receive an email on the process to access them and start the learning. This would be done within 1.5 weeks of the Deep skilling learning journey.

SME Connect Session

- We will have scheduled sessions with Subject Matter Experts (SMEs) that are designed to deepen your understanding of complex topics.
- **100% attendance is mandatory** for SME Connect sessions.
- Engage actively in doubt clarification sessions, asking questions and seeking clarification on challenging topics.
- Benefit from the experts’ experience and insights to gain a deeper understanding of the subject matter.

Disclaimer: Cognizant does not claim ownership or responsibility for the content or any issues with the links provided, as they are merely references available on the internet. Candidates are free to leverage additional sources beyond what has been provided to enhance their skill capabilities for Deep Skilling.

Effective Learning Strategies

Create a Study Schedule	Set Clear Goals	Stay Organized	Active Learning
Dedicate specific times each week for learning. Aim for 10-12 hours of study per week.	Define what you want to achieve each week. Break down tasks into manageable chunks.	Keep track of your progress and deadlines. Use tools like calendars, to-do lists, and reminders.	Engage with the material through hands-on exercises. Practice coding and solving problems regularly.

Duration Recommendation

Weekly Learning: 10-12 hours	Total Program Duration: 7 weeks
Allocate 1-2 hours daily or schedule longer sessions on weekends. Balance your learning with regular academic commitments.	Follow the recommendations consistently. Ensure to complete the mandatory exercises and review sessions.

Where to Practice?

Installed Software Tools
Install Python (latest version) from https://www.python.org/downloads/ Install Visual Studio Code from https://code.visualstudio.com/download Install Node.js (includes npm) from https://nodejs.org/en/download/ Install Git from https://git-scm.com/downloads Install PostgreSQL database from https://www.postgresql.org/download/ Install MySQL Community Server from https://dev.mysql.com/downloads/mysql/ Install MongoDB Community Server from https://www.mongodb.com/try/download/community Install MongoDB Compass (GUI tool) from https://www.mongodb.com/products/tools/compass Install Selenium IDE (browser extension) from https://www.selenium.dev/selenium-ide/

Exercise Instructions

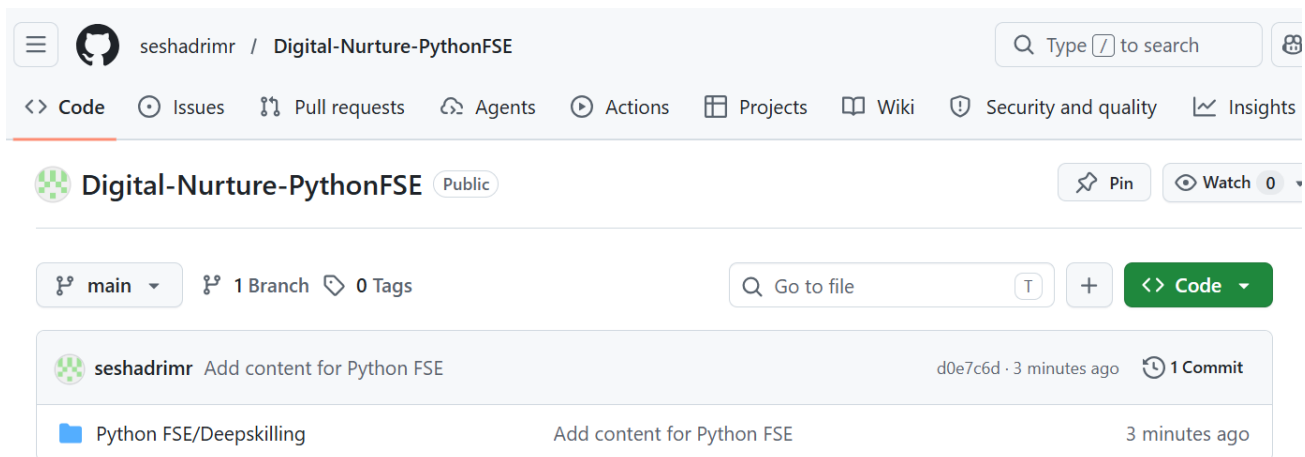
In this learning program, you will be required to complete weekly exercises designed to reinforce the concepts learned during the week. These exercises are hosted on a public GitHub repository and must be downloaded and solved on a weekly basis. Follow the instructions below to ensure a smooth and productive exercise workflow:

1. Accessing Exercises:

- Each week, exercises will be made available in our public GitHub repository.
- The repository URL is [this](#).

2. Downloading Exercises:

- You'll see the options 'Python FSE' as shown in the screenshot below. Click the 'Code' button as shown in the screenshot and choose the option 'Download zip'. This will download all the required Hands-on content for your practice.



3. Solving the Exercises:

- Solve the problem statements provided in the downloaded files.
- Ensure that you understand the problem requirements and apply the concepts learned during the week.
- Take your time to think through the solutions and code them accurately.

4. Self-Evaluation:

- After completing the exercises, evaluate your solutions based on the problem criteria.
- Compare your approach with any hints or solutions provided if available.
- Reflect on any mistakes or areas where you can improve.

5. Submitting Solutions:

- Firstly, organize your solutions week-wise and keep them in a folder.
- Create a **public repository** in your personal GitHub account, upload your solution folder, and share the URL with the POC on demand.

6. Additional Support:

- If you encounter difficulties or have questions about the exercises, seek help from your peers.
- Utilize the resources and links provided in this handbook for further assistance.

Upskilling skills – A quick recap

Hi Champ – Welcome to the Deep skilling of Digital Nurture 5.0 program. Before you delve deep into the skills into this, with utmost sincerity, please do go through the upskilling handbook and refresh the basic programming skills to keep yourself abridged.

Always remember – The Object-Oriented Programming concepts are the core for any Programming language or frameworks.

Correlate the concepts you learn with real-life to understand and relate better for better understanding and life-long knowledge retention.

All the best!

FSE construct - Engineering concepts

Engineering Concepts

The core concepts for any engineering landscape. This is the foundation on which all programming languages and frameworks are built.

Relevant skills:

- Design Patterns and Principles
- Data Structures and Algorithms

Please plan to complete the skills in 2 days.

Module 1 - Design Patterns and Principles

Overview:

This module introduces learners to essential design principles and patterns that are crucial for creating robust and maintainable software. Learners will delve into the SOLID principles, which include SRP, OCP, LSP, ISP, and DIP. They will also explore common design patterns, such as Creational, Structural, and Behavioral patterns, which provide reusable solutions to common design challenges. The module combines theoretical insights with practical exercises, helping learners understand and apply these concepts in their projects, ultimately enhancing their ability to write clean, efficient, and scalable code.

Learning Objectives:

After completing this module, users will be able to:

- Grasp the importance of the Single Responsibility Principle (SRP).
- Implement the Open/Closed Principle (OCP) to create extendable software.
- Ensure software components adhere to the Liskov Substitution Principle (LSP).
- Design interfaces following the Interface Segregation Principle (ISP).
- Apply the Dependency Inversion Principle (DIP) for flexible dependency management.
- Recognize and apply Creational Patterns.
- Utilize Structural Patterns to build robust systems.
- Employ Behavioral Patterns for effective object interaction.
- Improve code reusability and reduce duplication through design patterns.
- Solve common software design problems using appropriate design patterns.
- Evaluate and choose the right design pattern for a given problem context.

Self-Learning (Open-source links):

Key Topics	Sub-topics	Learning Reference Links
SOLID Principles	What & Why, The SOLID Principles of Object-Oriented Programming - The Single Responsibility Principle, The Open-Closed Principle, The Liskov Substitution Principle, The Interface Segregation Principle, The Dependency Inversion Principle	https://www.baeldung.com/solid-principles
Commonly used Design Patterns	GoF, Creational Patterns - Singleton Pattern, Factory Method Pattern, Builder Pattern; Structural Patterns - Adapter Pattern, Decorator Pattern, Proxy Pattern; Behavioral Patterns - Observer Pattern, Strategy Pattern, Command Pattern; Architectural Patterns - Model-View-Controller (MVC), Dependency Injection	https://medium.com/@softwaretechsolution/design-pattern-81ef65829de2

Hands-On:

- Complete the skills' hands-on exercises to reinforce your learning. Refer to the **Exercise Instructions** section above to access the practice content.

Check Your Understanding:

- <https://www.javaguides.net/2023/07/design-patterns-quiz.html>

Module 2 - Data Structures and Algorithms

Overview:

This module focuses on core data structures and algorithms, foundational for writing efficient and scalable code. Learners will explore key data structures such as arrays, linked lists, and understand their implementation and application scenarios. Additionally, they will delve into fundamental algorithms including searching (linear search, binary search) and sorting (bubble sort, quick sort, merge sort). The

module emphasizes practical implementation through coding exercises, enabling learners to apply these concepts effectively in real-world programming challenges.

Learning Objectives:

After completing this module, learners will be able to:

- Choose appropriate data structures based on problem requirements.
- Apply searching algorithms to find elements in data structures.
- Utilize sorting algorithms to order data efficiently.
- Solve coding problems using appropriate data structures and algorithms.
- Analyze algorithmic complexities to optimize performance.
- Design efficient and scalable solutions for software development tasks using learned concepts.

Self-Learning (Open-source links):

Key Topics	Sub-topics	Learning Reference Links
Analysis of Algorithms	Introduction, Why DS& Algorithm, Types of DS, Notations, Time and Space Complexity, Start using frameworks for describing and analyzing algorithms, Begin using asymptotic notation to express running-time analysis, Asymptotic notations for run-time analysis of algorithms, Best Case, Average Case, Worst case analysis of an algorithm, Finding Time Complexity of few iterative and recursive algorithms	https://www.geeksforgeeks.org/design-and-analysis-of-algorithms/
Sorting	Bubble, Insertion, Heap Sort, Quick Sort, Merge Sort - Worst, Average and Best-Case analysis	https://www.geeksforgeeks.org/sorting-algorithms/
Arrays	Array Traversal - Array representation in Memory, Measuring Time complexity, Searching, Traversal in Arrays, when to use Arrays	https://www.geeksforgeeks.org/array-data-structure-guide/
Searching	Linear Search, Binary Search	https://www.geeksforgeeks.org/searching-algorithms/#basics-of-searching-algorithms

Hands-On:

- Complete the respective skill's hands-on exercises to reinforce your learning. Refer to the **Exercise Instructions** section above to access the practice content.

Check Your Understanding:

- <https://www.proprofs.com/quiz-school/story.php?title=data-structures-algorithms-mcqs-objective-set-1>

FSE construct - Programming Languages

The languages are used to implement the necessary logic for the requirements.

Relevant skills:

- Python - Covered in upskilling
- SQL – Basic (covered in upskilling) and Advanced (covered in deepskilling) – 2 days
- Unit testing - PyTest, unittest, Jest & Mocha – 4 days

Please plan to complete this in 6 days.

Module 3 - Database Integration - PostgreSQL, MySQL & MongoDB

Overview:

This module covers a little advanced topics in SQL, MongoDB for programming in it, apply logic to formulate Stored procedures and execute them, transactions to ensure data integrity.

Learning Objectives:

- After completing this module, learners will be able to:
- Write and optimise SQL queries for relational databases
- Model and query a MongoDB document store
- Integrate a database into a Python backend using an ORM
- Design a schema with foreign keys; write JOIN queries; add indexes; read EXPLAIN output
- Create a collection; insert/query/update/delete documents; write a 2-stage aggregation pipeline
- Define ORM models; run migrations; perform CRUD via ORM; handle connection pooling

Self-Learning (Open-source links):

Key Topics	Sub-topics	Learning Reference Links
Advanced Concepts	Stored procedures, Triggers, Views, Transactions & locking, Replication concepts, Performance tuning	https://dev.mysql.com/doc/refman/8.0/en/tutorial.html https://www.w3schools.com/mysql/ https://www.geeksforgeeks.org/mysql-tutorial/
PostgreSQL	Installation & setup, Data types, Advanced queries, JSON support,	https://www.postgresql.org/docs/current/tutorial.html

	Stored procedures, Performance features	https://www.geeksforgeeks.org/postgresql-tutorial/ https://www.w3schools.com/postgresql/ https://www.tutorialspoint.com/postgresql/index.htm
MongoDB	Documents & collections, CRUD operations, Aggregation pipeline, Indexes, Replica sets, Sharding (concept)	https://www.mongodb.com/docs/manual/tutorial/getting-started/ https://www.w3schools.com/mongodb/ https://www.geeksforgeeks.org/mongodb-tutorial/ https://learn.mongodb.com/
Database Schema Design	Normalization (1NF, 2NF, 3NF), ER diagrams, Relationships, Denormalization trade-offs, NoSQL data modelling	https://www.geeksforgeeks.org/database-design-introduction/ https://vertabelo.com/blog/database-design-best-practices/ https://www.guru99.com/database-normalization.html
Query Optimization	Index types, EXPLAIN/ANALYZE, Query plans, N+1 query problem, Caching strategies, Connection pooling	https://www.geeksforgeeks.org/sql-performance-tuning/ https://use-the-index-luke.com/ https://www.postgresql.org/docs/current/using-explain.html
Migrations & Versioning	Migration concepts, Django Migrations, Alembic for SQLAlchemy, Flyway, Rollback strategies	https://docs.djangoproject.com/en/stable/topics/migrations/ https://alembic.sqlalchemy.org/en/latest/tutorial.html https://flask-migrate.readthedocs.io/en/latest/
ORMs & ODMs	SQLAlchemy, Django ORM, Mongoose (Node), Pros/cons of ORMs, Raw SQL when needed	https://docs.sqlalchemy.org/en/20/orm/quickstart.html https://docs.djangoproject.com/en/stable/topics/db/queries/ https://mongoosejs.com/docs/guide.html

Hands-On:

- Complete the respective skill's hands-on exercises to reinforce your learning. Refer to the **Exercise Instructions** section above to access the practice content.

Check Your Understanding:

PostgreSQL

- https://www.w3schools.com/postgresql/postgresql_quiz.php
- https://www.tutorialspoint.com/postgresql/quiz_on_postgresql_using_joins.htm

MongoDB

- <https://www.geeksforgeeks.org/mongodb/mongodb-online-quiz-test-your-nosql-knowledge/>

Others

- <https://www.geeksforgeeks.org/quizzes/introduction-to-database-design-quiz/>
- <https://www.geeksforgeeks.org/quizzes/sql-optimization/>
- <https://www.geeksforgeeks.org/quizzes/sql-indexes-quiz/>
- <https://www.geeksforgeeks.org/quizzes/views-indexes-triggers-quiz/>
- <https://www.propofs.com/quiz-school/quizzes/pp-stored-procedure-basics-quiz>

Module 4 – Unit Testing - PyTest, unittest, Jest & Mocha

Overview:

This module focuses on the Unit testing concepts through PyTest, unittest, Jest & Mocha.

Learning Objectives:

- After completing this module, learners will be able to:
- Write and organize unit tests using PyTest for a Python codebase
- Use unittest.mock to isolate external dependencies in tests
- Write unit and component tests for a JavaScript app using Jest
- Apply Test-Driven Development (TDD) red-green-refactor cycle
- Write ≥10 tests covering happy path & edge cases; use fixtures; generate a coverage report
- Mock a database call and an HTTP request in unit tests; verify mock was called with correct args
- Write tests for utility functions and a React component; mock an API call; run coverage report
- Implement a small feature (e.g. a calculator or validator) strictly following TDD cycle

Self-Learning (Open-source links):

Key Topics	Sub-topics	Learning Reference Links
Testing Fundamentals	Why test? Testing pyramid, Unit vs Integration vs E2E, Test design principles, Arrange-Act-Assert (AAA)	https://www.geeksforgeeks.org/software-engineering-software-testing-basics/ https://martinfowler.com/articles/practical-test-pyramid.html https://realpython.com/python-testing/
PyTest	Installation, Test discovery, Fixtures, Parameterization, Markers, Plugins, pytest-cov for coverage	https://docs.pytest.org/en/stable/getting-started.html https://realpython.com/pytest-python-testing/ https://www.geeksforgeeks.org/python/pytest-getting-started/ https://testdriven.io/blog/modern-tdd/
Python unittest	TestCase class, setUp/tearDown, Assertions, Test suites, Mocking with unittest.mock	https://docs.python.org/3/library/unittest.html https://realpython.com/python-unittest https://docs.python.org/3/library/unittest.mock.html
Jest (JavaScript)	Test setup, describe/test blocks, Matchers, Mocks & spies, Snapshot testing, Testing React components	https://jestjs.io/docs/getting-started https://www.w3schools.com/jest/ https://testing-library.com/docs/react-testing-library/intro/ https://jestjs.io/docs/tutorial-react
Mocha (JavaScript)	Test runner setup, BDD style with Chai, Assertions, Async testing, Hooks (before/after)	https://mochajs.org/ https://www.geeksforgeeks.org/unit-testing-using-mocha-and-chai/ https://www.chaijs.com/api/bdd/
Mocking & Test Doubles	Stubs, Mocks, Spies, Fakes, When to mock, Avoiding over-mocking	https://martinfowler.com/articles/mocksArentStubs.html https://realpython.com/python-mock-library/ https://sinonjs.org/
Test-Driven Development	Red-Green-Refactor cycle, Writing tests first, Refactoring with tests, Common TDD pitfalls	https://www.geeksforgeeks.org/test-driven-development-tdd/ https://martinfowler.com/bliki/TestDrivenDevelopment.html https://testdriven.io/test-driven-development/

Code Coverage & Quality	Coverage metrics (line, branch), coverage.py, Istanbul/nyc, Linting (ESLint, Pylint), Quality gates	https://coverage.readthedocs.io/ https://github.com/istanbuljs/nyc https://eslint.org/docs/latest/use/getting-started
-------------------------	---	---

Hands-On:

Complete the respective skill's hands-on exercises to reinforce your learning. Refer to the **Exercise Instructions** section above to access the practice content.

Check Your Understanding:

- https://www.tutorialspoint.com/pytest/quiz_on_pytest_introduction.htm
- <https://realpython.com/quizzes/python-unittest/>
- <https://www.geeksforgeeks.org/quizzes/testing-in-react/>
- <https://practice.expandtesting.com/mocha-chai-sinon-player>
- <https://www.proprofs.com/quiz-school/quizzes/pp-tdd-red-green-refactor-cycle-quiz>
- <https://www.freshersnow.com/pytest-mcqs-and-answers-with-explanation/>

FSE construct - Products and Frameworks

Products and Frameworks

There are numerous frameworks in the programming family to build applications. These enable the programmers to develop different types of applications by providing a gamut of features provided over a single integrated environment.

Please plan to complete this in 4 weeks' time.

Relevant skills:

- Python Backend Frameworks – Django, Flask & FastAPI – 4 days
- Frontend Development Basics – React, Angular & Vue.js overview – 9 days
- **Core skill:** QA Concepts & Test Automation – (Selenium Basics) – 6 days

Module 5 - Python Backend Frameworks – Django, Flask & FastAPI

Overview:

This module introduces the learner to the Python backend frameworks that are widely used.

Learning Objectives:

- After completing this module, learners will be able to:
Build and run a basic web server using Django, Flask, or FastAPI
- Design and expose a RESTful API with proper HTTP conventions
- Implement token-based authentication in a backend service
- Create a project from scratch; define URL routes; return JSON responses; run a dev server locally

- Build CRUD endpoints; handle request validation; return correct HTTP status codes
- Add JWT login/logout endpoints; protect routes; hash passwords with bcrypt/passlib

Self-Learning (Open-source links):

Key Topics	Sub-topics	Learning Reference Links
Web Framework Foundations	Web framework concepts, MVC/MVT pattern, Request-response cycle, WSGI/ASGI, Routing & middleware	https://www.geeksforgeeks.org/python/python-web-development-django/ https://realpython.com/tutorials/web-dev/ https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps
Django Framework	Project setup, Models & ORM, Views & Templates, URL routing, Forms, Admin interface, Authentication	https://docs.djangoproject.com/en/stable/intro/tutorial01/ https://www.geeksforgeeks.org/python-django/ https://realpython.com/get-started-with-django-1/ https://www.w3schools.com/django/
Flask Framework	App structure, Routing, Templates with Jinja2, Blueprints, Extensions, SQLAlchemy ORM	https://flask.palletsprojects.com/en/stable/quickstart/ https://www.geeksforgeeks.org/flask-tutorial/ https://realpython.com/tutorials/flask/ https://www.tutorialspoint.com/flask/index.htm
FastAPI Framework	Async/await, Path & query parameters, Pydantic models,	https://fastapi.tiangolo.com/tutorial/

		Dependency injection, OpenAPI/Swagger docs	https://www.geeksforgeeks.org/python/fastapi-introduction/ https://realpython.com/fastapi-python-web-apis/ https://testdriven.io/blog/fastapi-crud/
RESTful Design	API	REST principles, HTTP methods & status codes, Resource modelling, Versioning, Pagination, Filtering	https://www.geeksforgeeks.org/rest-api-introduction/ https://restfulapi.net/ https://www.redhat.com/en/topics/api/what-is-a-rest-api https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design
Microservices Architecture		Monolith vs Microservices, Service decomposition, Inter-service communication, API Gateway, Service discovery	https://www.geeksforgeeks.org/microservices/ https://microservices.io/patterns/microservices.html https://martinfowler.com/articles/microservices.html https://aws.amazon.com/microservices/
Authentication & Security		JWT tokens, OAuth2 flow, Session-based auth, Password hashing, CORS, Common vulnerabilities (OWASP Top 10)	https://www.geeksforgeeks.org/computer-networks/json-web-token-jwt/ https://oauth.net/2/ https://owasp.org/www-project-top-ten/ https://realpython.com/token-based-authentication-with-flask/

Hands-On:

- Complete the respective skill's hands-on exercises to reinforce your learning. Refer to the **Exercise Instructions** section above to access the practice content.

Check Your Understanding:

- https://www.w3schools.com/django/django_quiz.php
- <https://www.geeksforgeeks.org/quizzes/flask/>
- https://www.tutorialspoint.com/fastapi/quiz_on_fastapi_introduction.htm
- <https://www.geeksforgeeks.org/quizzes/restful-api/>
- <https://www.geeksforgeeks.org/quizzes/authentication-with-jwt-quiz/>
- https://www.tutorialspoint.com/oauth2.0/quiz_on_oauth2.0_overview.htm
- <https://www.javaguides.net/2023/01/microservices-quiz-mcq-multiple-choice.html>

Module 6 - Frontend Development Basics – React, Angular & Vue.js overview

Overview:

This module introduces the learner to the most widely used Front-end frameworks.

Learning Objectives:

- After completing this module, learners will be able to:
- Build and style a multi-section web page using HTML5 & CSS3
- Write interactive JavaScript using DOM manipulation and ES6+ syntax
- Build a simple single-page app using React.js
- Build a simple app using Angular
- Build a simple app using Vue.js
- Create a structured HTML page; apply Flexbox/Grid layout; make it mobile-responsive with media queries
- Select & modify DOM elements; handle click/submit events; fetch data from a public API asynchronously
- Create reusable components; manage local state; fetch and display API data; navigate between routes
- Scaffold a project with CLI; create components; bind data with [(ngModel)]; wire up a route
- Create a Vue 3 project; use reactive data; build a list/filter UI; add basic navigation

Self-Learning (Open-source links):

Key Topics	Sub-topics	Learning Reference Links
Web Foundations	HTML5 semantic elements, CSS3 (Flexbox, Grid), JavaScript ES6+ (let/const, arrow functions, promises, async/await, modules)	https://developer.mozilla.org/en-US/docs/Learn https://www.w3schools.com/html/ https://www.w3schools.com/css/ https://javascript.info/
React.js	JSX, Components, Props & State, Hooks (useState, useEffect, useContext), Routing (React Router), State management	https://react.dev/learn https://www.w3schools.com/react/ https://www.geeksforgeeks.org/reactjs-tutorials/

		https://reactrouter.com/start/library/installation
Angular	TypeScript basics, Components & Modules, Services & DI, RxJS, Routing, Reactive forms, HttpClient	https://angular.dev/tutorials https://www.w3schools.com/angular/ https://www.geeksforgeeks.org/angular-tutorial/ https://www.typescriptlang.org/docs/handbook/intro.html
Vue.js	Template syntax, Reactivity, Components, Composition API, Vue Router, Pinia/Vuex state management	https://vuejs.org/guide/introduction.html https://www.w3schools.com/vue/ https://www.geeksforgeeks.org/vue-js/ https://router.vuejs.org/guide/
Responsive Design	Mobile-first design, Media queries, Fluid layouts, CSS Grid & Flexbox, Viewport units, Container queries	https://www.w3schools.com/css/css_rwd_intro.asp https://web.dev/learn/design/ https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Responsive_Design
Web Accessibility (a11y)	WCAG 2.1 guidelines, ARIA attributes, Semantic HTML, Keyboard navigation, Screen reader support, Color contrast	https://www.w3.org/WAI/fundamentals/accessibility-intro/ https://web.dev/learn/accessibility/ https://developer.mozilla.org/en-US/docs/Web/Accessibility https://www.a11yproject.com/checklist/
Cross-Browser Compatibility	Browser engines, Feature detection, Polyfills, Vendor prefixes, Progressive enhancement, Testing tools	https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Cross_browser_testing https://caniuse.com/ https://www.browserstack.com/guide/cross-browser-testing
API Integration & State	Fetch API, Axios, Promises & async/await, Error handling, Redux/NgRx/Pinia for state management	https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API https://axios-http.com/docs/intro https://redux.js.org/tutorials/essentials/part-1-overview-concepts

Hands-On:

- Complete the respective skill's hands-on exercises to reinforce your learning. Refer to the **Exercise Instructions** section above to access the practice content.

Check Your Understanding:

- https://www.w3schools.com/htmlcss/htmlcss_quiz.asp
- https://www.w3schools.com/js/js_quiz.asp
- https://www.w3schools.com/REACT/react_quiz.asp
- https://www.w3schools.com/angular/angular_quiz.asp
- https://www.w3schools.com/vue/vue_quiz.php
- <https://www.freecodecamp.org/learn/responsive-web-design-v9/quiz-responsive-web-design/quiz-responsive-web-design>

Module 7 – QA Concepts & Test Automation – (Selenium Basics)

Overview:

This module focuses on the core skills for the Quality assurance (QA) requirements, core concepts of QA and test automation using Selenium skill.

Learning Objectives:

- After completing this module, learners will be able to:
- Explain the QA process and defect lifecycle in an SDLC context
- Differentiate SDLC and TDLC and apply the V-Model
- Design and implement a Selenium WebDriver automation script
- Apply the Page Object Model pattern to a Selenium test suite
- Write test cases for a given feature; classify defects by severity & priority; map tests to SDLC phases
- Map each SDLC phase to its corresponding TDLC phase; identify QA entry & exit criteria per phase
- Write a Python script that opens a browser, navigates to a URL, locates elements, and asserts expected text
- Refactor a flat script into POM; create ≥2 Page classes; call page methods from a separate test file

Self-Learning (Open-source links):

Skill Spring Recommended Course:

- Selenium Interactive Course

Key Topics	Sub-topics	Learning Reference Links
QA Concepts & Functional Testing	QA roles & responsibilities, Types of testing (functional, non-functional), Test levels, Black-box vs white-box testing, Defect lifecycle & management	https://www.geeksforgeeks.org/software-engineering-software-testing-basics/ https://www.guru99.com/software-testing.html https://www.tutorialspoint.com/software_testing/index.htm https://www.browserstack.com/guide/functional-testing

SDLC vs TDLC	Software Development Life Cycle phases, Test Development Life Cycle phases, V-Model, QA integration points in SDLC, Agile QA approach	https://www.geeksforgeeks.org/sdlc-software-development-life-cycle/ https://www.guru99.com/test-case-development-life-cycle-testing-process.html https://www.tutorialspoint.com/sdlc/sdlc_v_model.htm
Test Automation Process & Lifecycle	When to automate, Automation test lifecycle, Selecting test cases for automation, Framework types (linear, modular, data-driven, keyword-driven, hybrid), Automation ROI	https://www.geeksforgeeks.org/test-automation-in-software-testing https://www.guru99.com/automation-testing.html https://www.browserstack.com/guide/test-automation-framework https://www.atlassian.com/continuous-delivery/software-testing/automated-testing
Selenium Basics	Selenium architecture & components (WebDriver, Grid, IDE), Browser drivers setup, Locators (ID, Name, XPath, CSS), WebDriver commands, Handling waits (implicit/explicit), Running tests with Python	https://www.selenium.dev/documentation/webdriver/getting_started/ https://www.geeksforgeeks.org/selenium-python-tutorial/ https://www.guru99.com/selenium-tutorial.html https://realpython.com/modern-web-automation-with-python-and-selenium/
Selenium - Page Object Model (POM)	POM design pattern, Creating Page classes, Locator management, Separation of test logic from UI logic, Reusability & maintainability	https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/ https://www.guru99.com/page-object-model-pom-page-factory-in-selenium-ultimate-guide.html https://www.browserstack.com/guide/page-object-model-in-selenium

Hands-On:

- Complete the respective skill's hands-on exercises to reinforce your learning. Refer to the **Exercise Instructions** section above to access the practice content.

Check Your Understanding:

- <https://www.geeksforgeeks.org/quizzes/selenium-mcq/>
- <https://www.guru99.com/selenium-mcq-quiz.html>
- <https://www.sanfoundry.com/selenium-multiple-choice-questions-answers/>
- <https://www.javatpoint.com/selenium-mcq>
- <https://www.propofs.com/quiz-school/topic/selenium>

FSE construct – Agile, Code Review & Cross-Functional Collaboration, GenAI fundamentals

Platforms

These are the enablers as a base to host the application developed using the Engineering concepts, programming languages, product and frameworks to make it accessible by all users, reliable and scalable.

Relevant skills:

- Agile concepts
- GIT
- Cloud deployments
- GenAI fundamentals

Please plan to complete this in 1.5 weeks.

Module 8 – Agile, GIT

Overview:

This module focuses on the most widely used code repository tool, GIT.

Learning Objectives:

After completing this module, learners will be able to:

- Apply Scrum ceremonies and artefacts to a sprint simulation
- Follow professional Git branching and pull request workflows
- Give and receive constructive code review feedback
- Track and manage work using Agile project tools
- Break a feature into user stories with acceptance criteria; estimate with story points; simulate a sprint planning meeting
- Create a feature branch; make commits with meaningful messages; open a PR with description; resolve a merge conflict
- Review a peer's PR (or sample code) using a checklist; leave ≥3 comments (approvals, suggestions, blockers)
- Create a project board; add and assign issues; move cards through columns; close a sprint with summary

Self-Learning (Open-source links):

Key Topics	Sub-topics	Learning Reference Links
Agile Foundations	Agile Manifesto, Agile vs Waterfall, Iterative development, Roles in agile teams, Estimation techniques	https://www.atlassian.com/agile https://www.geeksforgeeks.org/software-engineering-agile-development-models/ https://agilemanifesto.org/ https://www.scrum.org/learning-series/what-is-agile
Scrum Framework	Scrum roles, Sprint planning, Daily standup, Sprint review, Retrospective, Product & sprint backlog	https://www.scrum.org/learning-series/what-is-scrum https://www.atlassian.com/agile/scrum https://www.geeksforgeeks.org/scrum-software-development-model/
Kanban	Kanban principles, Visualizing work, WIP limits, Cumulative flow, Continuous delivery	https://www.atlassian.com/agile/kanban https://www.geeksforgeeks.org/kanban-an-agile-methodology/ https://kanbanguides.org/english/
Git & GitHub Workflow	Git basics, Branching strategies, Pull requests, Merge vs rebase, Conflict resolution, Tags & releases	https://git-scm.com/book/en/v2 https://docs.github.com/en/get-started https://www.atlassian.com/git/tutorials https://learngitbranching.js.org/
Code Review Practices	What to look for in PRs, Giving constructive feedback, Reviewer/author etiquette, Review checklists	https://google.github.io/eng-practices/review/ https://www.geeksforgeeks.org/code-review-process/ https://github.com/joho/awesome-code-review https://medium.com/palantir/code-review-best-practices-19e02780015f
DevOps Collaboration	DevOps culture, Shift-left testing, Working with QA, Working with Ops, Incident response basics	https://www.atlassian.com/devops https://www.geeksforgeeks.org/devops-tutorial/ https://aws.amazon.com/devops/what-is-devops/

Project Tracking Tools	Jira basics, Azure Boards, GitHub Projects, User stories, Acceptance criteria, Story points	https://www.atlassian.com/software/jira/guides/getting-started/introduction https://learn.microsoft.com/en-us/azure/devops/boards/ https://docs.github.com/en/issues/planning-and-tracking-with-projects
------------------------	---	---

Hands-On:

Complete the respective skill's hands-on exercises to reinforce your learning. Refer to the **Exercise Instructions** section above to access the practice content.

Check your understanding:

- <https://worldofagile.com/free-agile-resources/agile-quiz/>
- https://www.tutorialspoint.com/scrum/quiz_on_scrum_estimation.htm
- https://www.w3schools.com/git/git_quiz.asp
- <https://www.geeksforgeeks.org/git/git-exercise/>
- <https://www.javaguides.net/2024/08/jira-quiz-mcq-questions-and-answers.html>

Module 9 – Cloud deployment

Overview:

This module introduces the fundamentals of Cloud, components for deployment, DevOps in widely used AWS, Azure and GCP.

Learning Objectives:

- After completing this module, learners will be able to:
- Deploy a containerised application to a cloud platform
- Configure a CI/CD pipeline that builds and deploys on code push
- Understand core cloud storage, compute, and managed DB services
- Write a Dockerfile; build & run container locally; deploy to a cloud free tier (Elastic Beanstalk or Cloud Run)
- Create a GitHub Actions YAML that runs tests, builds Docker image, and deploys to cloud on push to main
- Provision a free-tier resource (e.g. S3 bucket or Blob Storage); upload a file; set permissions via IAM role

Self-Learning (Open-source links):

Key Topics	Sub-topics	Learning Reference Links
AWS	EC2, S3, RDS, IAM, Elastic Beanstalk, Lambda (serverless), CloudWatch, AWS CLI	https://aws.amazon.com/getting-started/ https://www.geeksforgeeks.org/amazon-web-services-tutorial/ https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/Welcome.html

		https://www.tutorialspoint.com/amazon_web_services/index.htm
Microsoft Azure	App Service, Azure SQL, Blob Storage, Functions, Azure DevOps, Application Insights	https://learn.microsoft.com/en-us/training/azure/ https://www.geeksforgeeks.org/microsoft-azure-tutorial/ https://learn.microsoft.com/en-us/azure/app-service/overview https://learn.microsoft.com/en-us/azure/devops/
Google Cloud Platform	Compute Engine, Cloud Run, Cloud Storage, Cloud SQL, Cloud Functions, gcloud CLI	https://cloud.google.com/docs/get-started https://www.geeksforgeeks.org/google-cloud-platform-gcp-tutorial/ https://cloud.google.com/run/docs/quickstarts https://cloud.google.com/learn/training
Docker & Containerization	Docker concepts, Dockerfile, Images & containers, Docker Compose, Multi-stage builds, Best practices	https://docs.docker.com/get-started/ https://www.geeksforgeeks.org/devops/introduction-to-docker/ https://docker-curriculum.com/ https://docs.docker.com/compose/gettingstarted/
Kubernetes Basics	Pods, Deployments, Services, ConfigMaps, Secrets, kubectl basics, Managed K8s services (EKS/AKS/GKE)	https://kubernetes.io/docs/tutorials/kubernetes-basics/ https://www.geeksforgeeks.org/devops/kubernetes-tutorial/ https://learnk8s.io/learning
CI/CD Pipelines	Pipeline concepts, GitHub Actions workflows, Azure Pipelines, Build & deploy stages, Environment promotion Responsible use of Copilot – best practices	https://www.geeksforgeeks.org/devops/what-is-ci-cd/ https://docs.github.com/en/actions/learn-github-actions https://learn.microsoft.com/en-us/azure/devops/pipelines/get-started/ https://about.gitlab.com/topics/ci-cd/
Cloud Security & Monitoring	IAM principles, Secrets management, HTTPS/TLS, Logging & monitoring, Cost management basics	https://aws.amazon.com/iam/ https://learn.microsoft.com/en-us/azure/security/fundamentals/overview https://cloud.google.com/security/overview

Check your understanding:

- <https://www.w3schools.com/quiztest/quiztest.php?qtest=AWSCE>
- <https://www.geeksforgeeks.org/quizzes/amazon-web-services-quiz/>
- <https://www.geeksforgeeks.org/quizzes/microsoft-azure-mcq/>
- <https://www.geeksforgeeks.org/quizzes/google-cloud-platform-gcp-mcq/>
- <https://www.geeksforgeeks.org/quizzes/docker-mcq/>
- <https://www.geeksforgeeks.org/quizzes/kubernetes-mcq/>
- <https://www.geeksforgeeks.org/quizzes/devops-ci-cd-mcq/>
- <https://www.geeksforgeeks.org/quizzes/cloud-computing-security-mcq/>

Module 10 – Gen AI Fundamentals

Overview:

This module introduces the fundamentals of Generative AI along with practical usage of GitHub Copilot as an AI-powered coding assistant. It covers key concepts such as prompt engineering techniques, ethical considerations, and real-world applications of GenAI in software development. Learners will gain hands-on experience in setting up Copilot, leveraging its core features, and understanding the responsibilities and risks associated with AI-assisted coding.

Learning Objectives:

- After completing this module, learners will be able to:
- Explain the fundamentals, evolution, and applications of Generative AI
- Differentiate between generative AI and traditional AI approaches
- Apply prompt engineering techniques (zero-shot, few-shot, chain-of-thought) effectively
- Follow best practices and ethical guidelines for prompt design and AI usage
- Describe GitHub Copilot features, working principles, and supported environments
- Set up and configure GitHub Copilot in a development environment
- Use Copilot to generate code, documentation, and test cases efficiently
- Refactor and optimize code using AI-assisted suggestions
- Identify risks related to AI-generated code including security, licensing, and privacy concerns
- Apply responsible and secure practices while using AI coding tools

Self-Learning (Open-source links):

Key Topics	Sub-topics	Learning Reference Links
Introduction to Generative AI	What is Generative AI? How GenAI differs from traditional (discriminative) AI Overview of GenAI applications: text generation, code completion, image	Skill spring course FUNDAMENTALS OF GENERATIVE AI https://www.geeksforgeeks.org/artificial-intelligence/what-is-generative-ai/

	creation, chatbots History and evolution of Generative AI: • Early chatbots (1960s) → GANs (2014) → GPT-3 (2020) → ChatGPT (2022) → GitHub Copilot and beyond	https://www.geeksforgeeks.org/what-is-generative-ai/ https://businessmanagementblog.com/history-of-generative-ai/
Context engineering	What is Context Engineering and how it improves AI output quality Difference between Prompt Engineering and Context Engineering Types of Context in AI (instructions, user input, history, external data) Context Window concept and its limitations in LLMs • Importance of selecting relevant context to improve accuracy and reduce errors	https://www.datacamp.com/blog/context-engineering https://www.simplilearn.com/what-is-context-engineering-article https://www.kdnuggets.com/a-gentle-introduction-to-context-engineering-in-llms
Prompt Engineering – Techniques, Best Practices & Ethics	What is Prompt Engineering and why it matters for developers Prompting Techniques: • Zero-shot prompting – direct task without examples • Few-shot prompting – giving examples to guide the model • Chain-of-thought prompting – asking model to reason step by step Best practices: clear instructions, provide context, specify output format, iterate Ethical considerations: avoiding bias in prompts, accuracy, privacy, responsible AI usage Hands-on: writing coding-task prompts (e.g., "Write a C# method that...")	Skill spring course Prompt Engineering Foundation
Introduction to GitHub Copilot	What is GitHub Copilot? Overview of GitHub Copilot features How Copilot works (AI pair programmer powered by OpenAI Codex) Supported IDEs and languages	Skill spring course GitHub Copilot Foundation Virtual Training Course

Setup and Configuration	Installing GitHub Copilot extension in VS Code Connecting to a GitHub account Beginner-friendly first coding task with Copilot	https://www.freecodecamp.org/news/how-to-use-github-copilot-with-visual-studio-code/ https://www.geeksforgeeks.org/git/github-copilot/
Core Features and Capabilities	Code suggestions and completions (Tab to accept) Writing functions and boilerplate code from comments Generating comments and documentation automatically Refactoring and optimizing existing code Creating test cases with Copilot	https://www.geeksforgeeks.org/git/github-copilot/ https://www.freecodecamp.org/news/how-to-use-github-copilot-with-visual-studio-code/
Security and Ethical Considerations	Understanding AI-generated code risks (vulnerabilities, hallucinations) Licensing and attribution concerns (copyright risk) Data privacy and usage policies (what code is sent to GitHub servers) Responsible use of Copilot – best practices	https://blog.gitguardian.com/github-copilot-security-and-privacy/ https://www.geeksforgeeks.org/git/github-copilot/ https://docs.github.com/en/copilot/responsible-use/code-review
Agentic AI foundation	Basics of Agentic AI – core concepts, lifecycle Core principles of modern AI, including machine learning, neural networks, and real-world applications.	Skill spring course Introduction to Agentic AI Foundations of Modern AI

Check your understanding:

- <https://www.mygreatlearning.com/blog/generative-ai-quiz/>
- <https://aitoolsnote.com/quiz-prompt-engineering-mcq/>
- <https://technicalblog.in/languagewise-mcqs/prompt-engineering-mcqs/>
- <https://examsland.com/free-practice-test/github-copilot/>

What's Next?

Congratulations on successfully completing the 7-week DN 5.0 Deep Skilling learning program!

As you have now finished this important phase of your learning, you will be taking a **Knowledge-Based Assessment (KBA)** to certify your skills. **This assessment will cover all the skills and topics you have learnt in this tenure**, ensuring you have a comprehensive understanding of the material.

We wish you the best of luck with your assessment and look forward to seeing you apply your newly acquired knowledge and skills. Good luck!